



Politechnika Warszawska

Wydział Matematyki
i Nauk Informatycznych

Metody Sztucznej Inteligencji 2

Zastosowanie Monte-Carlo Tree Search do
implementacji sztucznej inteligencji w grze
Breakthrough

Raport końcowy

Zespół:

inż. Adam Grącikowski

inż. Piotr Iśtok

Prowadzący:

mgr inż. Bartłomiej Olechno

prof. dr hab. inż. Jacek Mańdziuk

Oświadczamy, że niniejsza praca, stanowiąca podstawę do uznania osiągnięcia efektów uczenia się z przedmiotu *Metody Sztucznej Inteligencji 2*, została wykonana samodzielnie.

Warszawa, 31 maja 2026

Spis treści

1 Instrukcja użytkownika	2
1.1 Środowisko i proces budowania	2
1.2 Opis plików wykonywalnych	2
1.2.1 Środowisko graficzne	2
1.2.2 Środowisko turniejowe	3
1.3 Format pliku wejściowego	3
1.3.1 Agent Minimax	4
1.3.2 Agent MCTS	4
1.3.3 Gracz ludzki	5
1.4 Format pliku wyjściowego	5
1.5 Interfejs graficzny	7
2 Weryfikacja hipotez badawczych	9
2.1 Hipoteza 1: Wpływ algorytmu RAVE	9
2.1.1 Metodologia i przebieg eksperymentu	9
2.1.2 Analiza wyników	9
2.1.3 Wnioski końcowe	10
2.2 Hipoteza 2: Wpływ algorytmu Heavy Playouts	11
2.2.1 Metodologia i przebieg eksperymentu	11
2.2.2 Analiza wyników	11
2.2.3 Wnioski końcowe	12
2.3 Hipoteza 3: Przewaga nad ewaluacją statyczną	13
2.3.1 Metodologia i przebieg eksperymentu	13
2.3.2 Analiza wyników	13
2.3.3 Wnioski końcowe	17
2.4 Hipoteza 4: Wpływ rozmiaru planszy na asymetrię szans	18
2.4.1 Metodologia i przebieg eksperymentu	18
2.4.2 Analiza wyników	18
2.4.3 Wnioski końcowe	20
3 Testy z udziałem ludzi	21
3.1 Metodologia i przebieg badania	21
3.2 Analiza wyników	21
3.3 Długość partii i zachowanie materiału	23
3.4 Ocena ankietowa i obserwacje jakościowe	24
3.5 Wnioski końcowe	24
4 Podsumowanie	26

1 Instrukcja użytkownika

1.1 Środowisko i proces budowania

Warstwa obliczeniowa oraz interfejs graficzny projektu zostały w całości zrealizowane w języku Rust [1]. Do poprawnego zbudowania i uruchomienia plików wykonywalnych niezbędne jest posiadanie dedykowanych dla tego języka narzędzi (ang. *Rust toolchain*). Zalecana wersja kompilatora to 1.88.0 lub nowsza.

Aby zweryfikować poprawność instalacji środowiska, należy wywołać w wierszu poleceń polecenia sprawdzające wersje kompilatora `rustc` [2] oraz menedżera pakietów `Cargo` [3] (Rysunek 1):

```
> rustc --version
rustc 1.88.0 (6b00bc388 2025-06-23)
> cargo --version
cargo 1.88.0 (873a06493 2025-05-10)
```

Rysunek 1: Weryfikacja instalacji środowiska Rust.

Aplikację należy budować w trybie optymalizacji kompilatora (ang. *release*) korzystając z terminala w głównym katalogu projektu (Rysunek 2):

```
> cd src/compute && cargo build --release
```

Rysunek 2: Budowanie aplikacji w trybie optymalizacji kompilatora.

Po udanej kompilacji, w katalogu `src/compute/target/release` wygenerowane zostaną dwa niezależne pliki wykonywalne: `breakthrough` oraz `tournament`. Każdy z programów umożliwia obsługę argumentów linii poleceń (CLI). Wbudowany system pomocy dla każdego z algorytmów można wywołać dodając flagę `-h` lub `--help` podczas uruchamiania z terminala (np. `./as --help`).

Wszystkie programy przyjmują na wejściu ścieżkę do pliku konfiguracyjnego w formacie TOML.

1.2 Opis plików wykonywalnych

W następnych podrozdziałach przedstawiono szczegółowy opis argumentów pozycyjnych, flag oraz opcjonalnych parametrów sterujących dla każdego z plików wykonywalnych.

1.2.1 Środowisko graficzne

Plik wykonywalny `breakthrough.exe` uruchamia graficzne środowisko testowe przeznaczone do pojedynczej rozgrywki pomiędzy ludźmi lub człowiekiem i agentem (jednym z zaimplementowanych algorytmów). Pozwala na zdefiniowanie wymiarów planszy, ustawienie ziarna losowości oraz zapis szczegółowych informacji z przebiegu partii osobno dla każdego z graczy (Rysunek 3).

```
> breakthrough.exe [OPTIONS] <CONFIG>
```

Rysunek 3: Podstawowe użycie graficznego środowiska testowego.

Tabela 1: Argumenty i opcje dla programu breakthrough.

Opcja / Argument	Opis
<CONFIG>	Ścieżka do pliku konfiguracyjnego TOML. Pozycja opcjonalna, domyślnie przyjmuje wartość <code>config.toml</code> .
--white-output <PATH>	(Opcjonalne) Ścieżka do pliku wynikowego JSONL z zapisem ruchów gracza białego. Domyślnie: <code>white_output.jsonl</code> .
--black-output <PATH>	(Opcjonalne) Ścieżka do pliku wynikowego JSONL z zapisem ruchów gracza czarnego. Domyślnie: <code>black_output.jsonl</code> .
--board-width <INT>	(Wymagane) Szerokość planszy do gry (liczba kolumn).
--board-height <INT>	(Wymagane) Wysokość planszy do gry (liczba wierszy).
--seed <INT>	(Opcjonalne) Ziarno dla generatora liczb losowych, kluczowe dla zapewnienia pełnej powtarzalności eksperymentów.
-h, --help	Wyświetla ekran pomocy z opisem wszystkich dostępnych parametrów.
-V, --version	Wyświetla wersję kompilacji programu.

1.2.2 Środowisko turniejowe

Plik wykonywalny `tournament.exe` pozwala na przeprowadzenie rozgrywki pomiędzy dwoma agentami (algorytmami). Interfejs wiersza poleceń zachowuje pełną spójność z graficznym środowiskiem testowym (Rysunek 4).

```
> tournament.exe [OPTIONS] <CONFIG>
```

Rysunek 4: Użycie modułu turniejowego.

Tabela 2: Argumenty i opcje dla programu tournament.

Opcja / Argument	Opis
<CONFIG>	Ścieżka do pliku konfiguracyjnego TOML. Pozycja opcjonalna, domyślnie przyjmuje wartość <code>config.toml</code> .
--white-output <PATH>	(Opcjonalne) Ścieżka do pliku wynikowego JSONL dla gracza białego. Domyślnie: <code>white_output.jsonl</code> .
--black-output <PATH>	(Opcjonalne) Ścieżka do pliku wynikowego JSONL dla gracza czarnego. Domyślnie: <code>black_output.jsonl</code> .
--board-width <INT>	(Wymagane) Szerokość planszy turniejowej.
--board-height <INT>	(Wymagane) Wysokość planszy turniejowej.
--seed <INT>	(Opcjonalne) Ziarno generatora liczb losowych używane do deterministycznej inicjalizacji partii w ramach turnieju.
-h, --help	Wyświetla ekran pomocy programu.
-V, --version	Wyświetla wersję kompilacji programu.

1.3 Format pliku wejściowego

Konfiguracja parametrów agentów biorących udział w rozgrywce odbywa się za pomocą pliku tekstowego w formacie TOML. W pliku tym definiuje się niezależne ustawienia dla obu stron (np. dla gracza białego i czarnego). Każdy z graczy musi mieć jawnie określony typ algorytmu sterującego (pole `type`), co determinuje listę dostępnych parametrów opcjonalnych.

W przypadku pominięcia któregośkolwiek z opcjonalnych pól, środowisko przyjmie predefiniowane wartości domyślne. Przykład struktury takiego pliku przedstawiono na Rysunku 5.

```

board_width = 6
board_height = 7
seed = 100

[white_player]
type = "Minimax"
max_depth = 5
material_weight = 250
advancement_weight = 10

[black_player]
type = "Mcts"
max_iterations = 100000
exploration_constant = 1.41
use_heavy_playouts = true

```

Rysunek 5: Przykładowy plik konfiguracyjny `config.toml`.

W następnych podrozdziałach opisano szczegółowo dostępne parametry dla każdego z trzech zaimplementowanych rodzajów graczy.

1.3.1 Agent Minimax

Wymaga podania parametrów sterujących przeszukiwaniem drzewa gry oraz wag poszczególnych komponentów funkcji heurystycznej oceniającej stan planszy.

Tabela 3: Pola konfiguracyjne dla agenta typu Minimax.

Pole	Domyślnie	Opis
<code>max_depth</code>	4	Maksymalna głębokość przeszukiwania drzewa gry.
<code>material_weight</code>	200	Waga heurystyki odpowiadająca za przewagę materialną (liczbę bierek).
<code>advancement_weight</code>	5	Waga heurystyki premiująca zaawansowanie pionów w kierunku linii docelowej przeciwnika.
<code>defended_weight</code>	5	Waga heurystyki promująca bezpieczne ustawienia, w których piony wzajemnie się bronią.
<code>edge_penalty_weight</code>	-2	Waga (kara) nakładana za umieszczenie pionów na krawędziach planszy.

1.3.2 Agent MCTS

Agent udostępnia on szereg zaawansowanych parametrów pozwalających na włączenie dodatkowych rozszerzeń, takich jak heurystyka RAVE czy Heavy Playouts.

Tabela 4: Pola konfiguracyjne dla agenta typu MCTS.

Pole	Domyślnie	Opis
max_iterations	75000	Maksymalna liczba iteracji algorytmu MCTS (liczba rozbudów drzewa).
max_time_ms	None	Opcjonalny, twardy limit czasu obliczeń dla jednego ruchu w milisekundach. Jeśli jest zdefiniowany, nadpisuje warunek liczby iteracji.
exploration_constant	1.41	Stała eksploracji (C) w formule UCT, balansująca proces poszukiwań.
use_rave	false	Wartość logiczna aktywująca modyfikację RAVE.
rave_k	1000.0	Parametr k sterujący wagą heurystyki RAVE w stosunku do standardowych wartości UCT.
use_heavy_playouts	false	Wartość logiczna włączająca ulepszone symulacje w fazie <i>rollout</i> , kierowane funkcją oceny planszy.
heavy_playouts_epsilon	0.1	Prawdopodobieństwo wykonania ruchu losowego (ϵ) w ulepszonych symulacjach (strategia ϵ -greedy).
material_weight	200	Waga przewagi materialnej, wykorzystywana przez funkcję oceny w fazie <i>heavy playouts</i> .
advancement_weight	5	Waga zaawansowania pionów dla symulacji <i>heavy playouts</i> .
defended_weight	5	Waga bronionych pionów dla symulacji <i>heavy playouts</i> .
edge_penalty_weight	-2	Waga (kara) za piony na krawędzi dla symulacji <i>heavy playouts</i> .

1.3.3 Gracz ludzki

Opcja dedykowana do manualnego testowania środowiska. Pozwala na zdefiniowanie interaktywnego gracza ludzkiego (`type = "Human"`), który wprowadza swoje ruchy poprzez interfejs graficzny GUI. Ten wariant agenta nie wymaga i nie przyjmuje żadnych dodatkowych parametrów konfiguracyjnych.

1.4 Format pliku wyjściowego

Wyniki działania środowiska testowego oraz turniejowego zapisywane są w plikach tekstowych o strukturze JSONL (JSON Lines). Każda linia w takim pliku stanowi niezależny, poprawny obiekt JSON, który agreguje statystyki i parametry z perspektywy pojedynczego gracza w rozegranej partii.

Struktura generowanego logu zawiera zestaw parametrów wspólnych (rejestrowanych dla wszystkich typów graczy, w tym człowieka) oraz sekcję pól specyficznych, które zależą od użytego algorytmu. Przykładowy zrzut danych wyjściowych dla agenta wykorzystującego algorytm Minimax przedstawiono na Rysunku 6.

```

{
  "board_width": 6,
  "board_height": 7,
  "seed": 100,
  "agent_type": "Minimax",
  "agent_color": "Black",
  "agent_won": true,
  "pieces_remaining": 10,
  "move_times_ms": [32, 74, 93, ...],
  "total_moves": 16,
  "moves": ["a6a5", "e6d5", "c6d5", ...],
  "opponent_type": "Mcts",
  "opponent_pieces_remaining": 6,
  "max_depth": 4,
  "total_nodes_evaluated": 33229,
  "total_cutoffs": 10248,
  "material_weight": 200,
  "advancement_weight": 5,
  "defended_weight": 5,
  "edge_penalty_weight": -2
}

```

Rysunek 6: Przykładowy rekord w pliku wyjściowym dla agenta typu Minimax.

W Tabeli 5 zestawiono atrybuty bazowe, które są generowane dla każdego uczestnika rozgrywki (w tym dla agenta typu Human).

Tabela 5: Wspólne pola wyjściowe logowane dla każdego gracza.

Pole	Opis
board_width, board_height	Wymiary planszy, na której rozegrano partię.
seed	Ziarno losowości wykorzystane w danej grze.
agent_type	Typ agenta generującego log (Minimax, Mcts lub Human).
agent_color	Kolor przypisany agentowi w partii (White lub Black).
agent_won	Flaga logiczna informująca, czy dany agent wygrał rozgrywkę.
pieces_remaining	Liczba pionów gracza, które pozostały na planszy w momencie zakończenia gry.
move_times_ms	Lista zawierająca czas obliczania każdego kolejnego ruchu przez agenta (w milisekundach).
total_moves	Całkowita liczba ruchów wykonana przez agenta.
moves	Sekwencja wszystkich wykonanych ruchów (np. "a6a5").
opponent_type	Typ agenta przeciwnika.
opponent_pieces_remaining	Liczba pionów przeciwnika na koniec gry.

W przypadku agentów zautomatyzowanych, bazowy zestaw danych jest rozszerzany o metryki charakterystyczne dla danego modułu heurystycznego. Tabela 6 przedstawia logowane parametry dla algorytmu Minimax, natomiast Tabela 7 dla algorytmu MCTS.

Tabela 6: Pola specyficzne dla agenta Minimax.

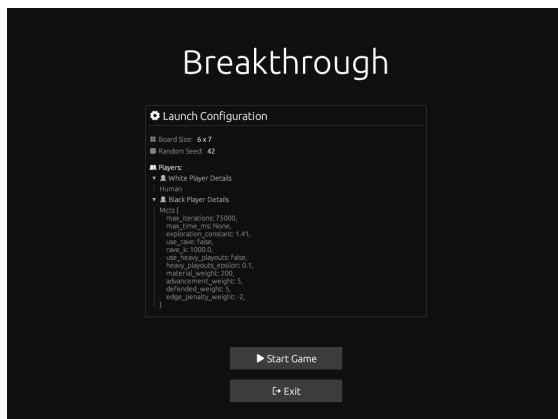
Pole	Opis
max_depth	Skonfigurowana maksymalna głębokość przeszukiwania.
total_nodes_evaluated	Łączna liczba wywołań funkcji statycznej oceny stanu gry (odwiedzonych liści) w trakcie całej partii.
total_cutoffs	Łączna liczba odcięć uzyskanych w całym drzewie przeszukiwań.
material_weight ...	Zestaw wag wykorzystanych w funkcji oceny (material, advancement, defended, edge_penalty).

Tabela 7: Pola specyficzne dla agenta MCTS.

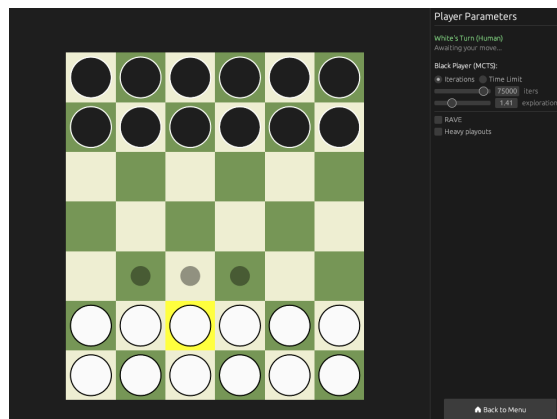
Pole	Opis
<code>max_iterations</code>	Limit liczby iteracji algorytmu MCTS.
<code>max_time_ms</code>	Limit czasu obliczeń dla jednego ruchu (lub <code>null</code> , jeśli brak).
<code>exploration_constant</code>	Wartość współczynnika eksploracji.
<code>use_rave, rave_k</code>	Informacja o wykorzystaniu heurystyki RAVE i jej parametrze k .
<code>use_heavy_playouts ...</code>	Flaga logiczna i parametry (<code>epsilon</code> oraz wagi heurystyki) wykorzystywane do kierowania fazą symulacji (rollout).
<code>total_iterations</code>	Zsumowana liczba wszystkich iteracji MCTS wykonanych przez agenta we wszystkich ruchach danej partii.
<code>total_nodes_created</code>	Sumaryczna liczba wygenerowanych w pamięci węzłów drzewa MCTS podczas gry.

1.5 Interfejs graficzny

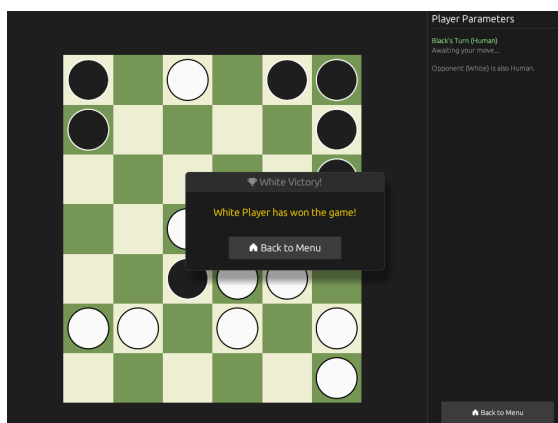
Aplikacja została wyposażona w prosty, czytelny interfejs graficzny (GUI), umożliwiający rozgrywkę człowieka z agentem. Zestawienie poszczególnych widoków aplikacji przedstawiono na Rysunku 7.



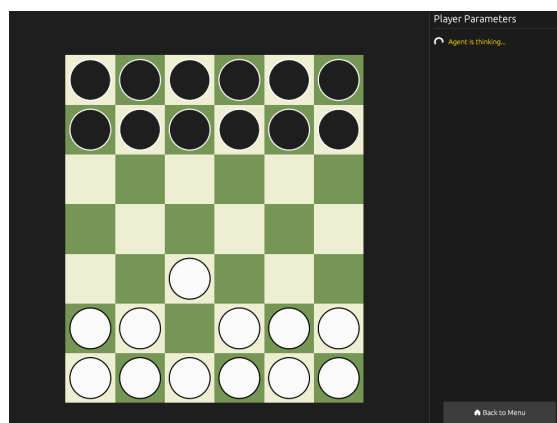
(a) Ekran startowy.



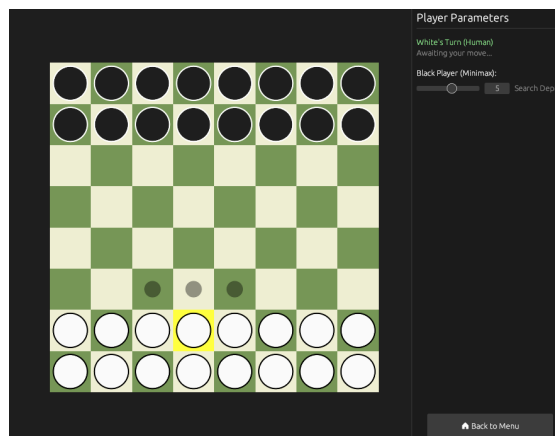
(b) Oczekiwanie na ruch gracza.



(c) Ekran końcowy z ogłoszeniem zwycięzcy.



(d) Oczekiwanie na ruch agenta.



(e) Gra z agentem Minimax.

Rysunek 7: Zrzuty ekranu interfejsu graficznego dla gry Breakthrough.

2 Weryfikacja hipotez badawczych

2.1 Hipoteza 1: Wpływ algorytmu RAVE

2.1.1 Metodologia i przebieg eksperymentu

Pierwsza hipoteza badawcza dotyczyła bezpośredniego porównania klasycznego algorytmu MCTS z jego rozszerzeniem wykorzystującym heurystykę RAVE (ang. *Rapid Action Value Estimation*). Celem eksperymentu było sprawdzenie, czy w warunkach stałego budżetu obliczeniowego, zdefiniowanego jako limit iteracji, wariant MCTS + RAVE osiągnie przewagę nad MCTS. Kryterium przyjęcia hipotezy było przekroczenie progu 55% zwycięstw przez agenta MCTS + RAVE w bezpośrednim starciu z agentem MCTS.

W każdej rozgrywce obaj agenci korzystali z identycznego limitu iteracji równego 10 000. Na wstępie dostrojono hiperparametr k algorytmu RAVE. Z testów wynikało, że wartość $k = 100$ dawała lepsze wyniki niż wartość domyślna $k = 1000$. Parametr k kontroluje tempo zaniku wpływu statystyk AMAF zgodnie ze wzorem:

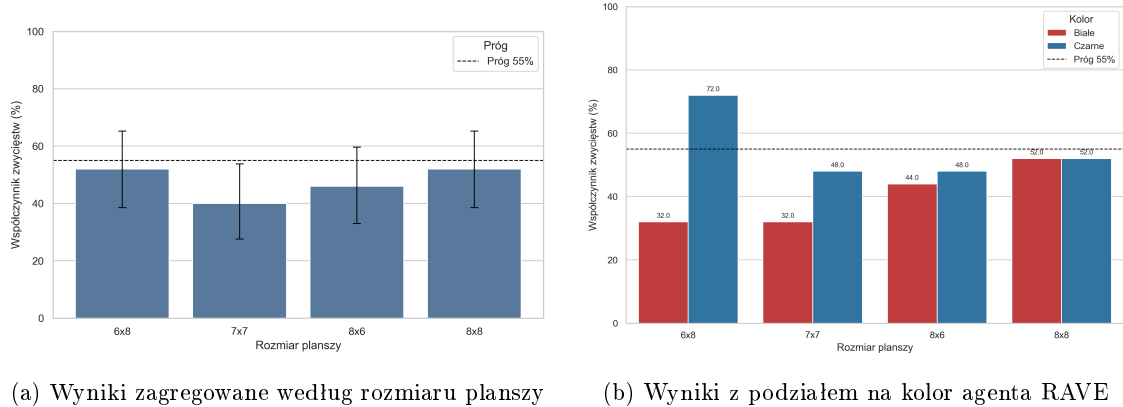
$$\beta = \frac{k}{k + n}$$

gdzie n oznacza liczbę odwiedzin rozpatrywanego węzła potomnego. Mniejsza wartość k powoduje zatem szybsze przejście od wartości RAVE do klasycznych statystyk UCT.

Eksperyment przeprowadzono na czterech rozmiarach planszy: 6×8 , 7×7 , 8×6 oraz 8×8 . Dla każdego rozmiaru planszy rozegrano 50 partii: 25 z agentem MCTS + RAVE jako graczem białym oraz 25 z agentem MCTS + RAVE jako graczem czarnym. Rotacja kolorów została zastosowana w celu ograniczenia wpływu przewagi pierwszego ruchu. Jak pokazują dalsze wyniki, kolor kontrolowanych pionów miał zauważalny wpływ na rezultaty. Łącznie w ramach weryfikacji hipotezy H1 rozegrano 200 partii.

2.1.2 Analiza wyników

Zestawienie współczynnika zwycięstw agenta MCTS + RAVE względem klasycznego MCTS przedstawiono na Rysunku 8. Pierwszy wykres agreguje wyniki według rozmiaru planszy, natomiast drugi dodatkowo rozdziela rezultaty ze względu na kolor kontrolowany przez agenta RAVE. Pozioma linia oznacza próg 55% zwycięstw, wymagany do pozytywnej weryfikacji hipotezy.



Rysunek 8: Współczynnik zwycięstw agenta MCTS + RAVE w starciu z klasycznym MCTS przy limicie 10 000 iteracji na ruch.

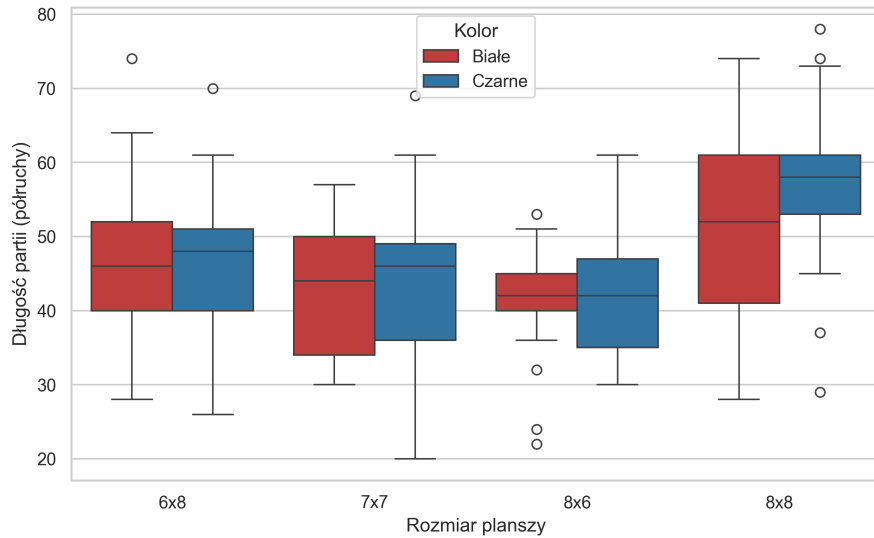
Wyniki liczbowe przedstawiono w Tabeli 8. W żadnym z badanych rozmiarów planszy średni wynik MCTS + RAVE nie przekroczył progu 55%. Największą skuteczność agent uzyskiwał na planszach 6×8 oraz 8×8 , gdzie wariant RAVE wygrał 52.0% partii. Najśłabszy rezultat odnotowano na planszy 7×7 , gdzie współczynnik zwycięstw wyniósł 40.0%.

Tabela 8: Wyniki eksperymentu H1: MCTS + RAVE przeciwko klasycznemu MCTS.

Rozmiar planszy	Zwycięstwa RAVE	Skuteczność	Średnia długość partii
6×8	26/50	52.0%	46.7
7×7	20/50	40.0%	43.5
8×6	23/50	46.0%	42.1
8×8	26/50	52.0%	54.4
Łącznie	95/200	47.5%	46.7

Sumaryczny wynik MCTS + RAVE wyniósł 95 zwycięstw w 200 partiach, co odpowiada skuteczności 47.5%. Przedział ufności Wilsona na poziomie 95% wynosił [40.7%, 54.4%], a więc nawet jego górna granica pozostaje poniżej progu 55%.

Dodatkowych wniosków dostarcza analiza rozkładu długości partii, przedstawiona na Rysunku 9.



Rysunek 9: Rozkład długości partii (w półruchach) w eksperymencie H1 z podziałem na rozmiar planszy i kolor.

Rozkład liczby rozegranych półruchów wykazuje silną zależność od fizycznych proporcji planszy. Najkrótsze partie odnotowano na planszy 8×6 (średnio 42.1 półruchów). Wynika to z faktu, że odległość do końcowej linii przeciwnika wynosi tylko 6 rzędów, co naturalnie skraca rozgrywkę. Z kolei na planszy 8×8 , wymagającej pokonania najdłuższego dystansu przy zachowaniu dużej szerokości frontu, gry były zauważalnie dłuższe (średnio 54.4 półruchów), a rozrzut wyników był największy. Zaskakującą obserwacją jest to, że w trzech z czterech rozmiarów planszy agent RAVE, grając po stronie czarnej, wykonywał średnio więcej ruchów niż grając po stronie białej. Może być to związane ze zwiększoną liczbą zwycięstw odnoszonych po stronie czarnej, choć na pierwszy rzut oka jest to sprzeczne z intuicją: gracz biały, rozpoczynając partię, ma możliwość wykonania jednego ruchu więcej od przeciwnika. Poza tą obserwacją dla badanego wariantu nie zaobserwowano jednoznacznych różnic w długości rozgrywki pomiędzy dwoma kolorami kontrolowanych pionów.

2.1.3 Wnioski końcowe

Hipoteza H1 zostaje odrzucona. Dla limitu 10 000 iteracji na ruch agent MCTS + RAVE, nawet po dobraniu korzystniejszego parametru $k = 100$, nie osiągnął wymaganego progu 55% zwycięstw w bezpośrednim starciu z bazowym MCTS. Wynik sugeruje, że w badanej implementacji i dla przyjętych rozmiarów planszy informacja AMAF nie poprawia stabilnie jakości decyzji względem klasycznego MCTS. Możliwym wyjaśnieniem jest fakt, że w grze Breakthrough kolejność ruchów oraz tempo dojścia do linii końcowej mają duże znaczenie taktyczne. Dodatkowo przypadkowe ruchy wykonywane w fazie *playout* mogą nie nieść wystarczająco użytecznej informacji AMAF, przez co założenie RAVE o użyteczności ruchu niezależnie od momentu jego wykonania okazuje się w tym środowisku zbyt silne i może prowadzić do przeszukiwania nieoptymalnych gałęzi drzewa gry.

2.2 Hipoteza 2: Wpływ algorytmu Heavy Playouts

2.2.1 Metodologia i przebieg eksperymentu

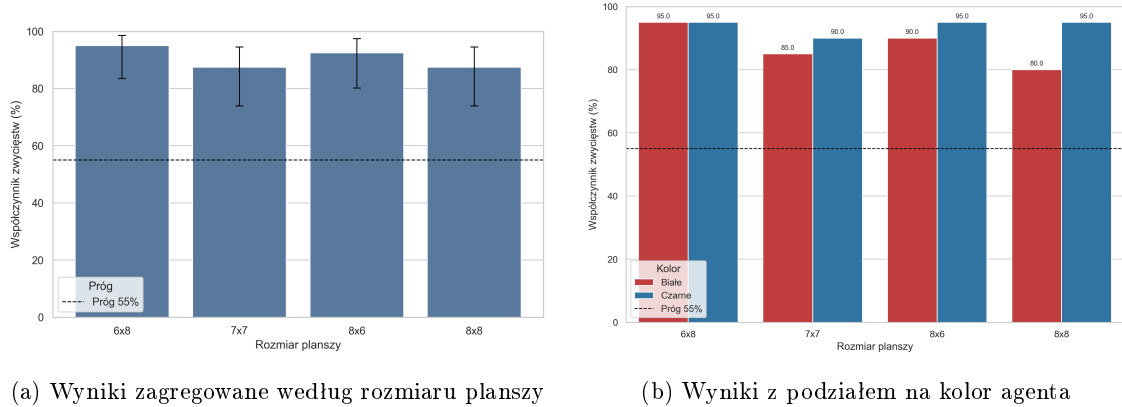
Druga hipoteza badawcza dotyczyła wpływu ukierunkowania fazy *playout* zgodnie z przyjętą funkcją heurystyczną, na jakość decyzji podejmowanych przez agenta MCTS. W odróżnieniu od eksperymentu H1, w tej serii pomiarowej nie narzucono stałej liczby iteracji. Wynika to z faktu, że pojedyncza iteracja agenta Heavy Playouts jest znacznie wolniejsza, ponieważ symulacja rozgrywki od liścia drzewa wymaga dodatkowej oceny pozycji, a nie jedynie wyboru losowego legalnego ruchu. Z tego powodu zastosowano limit czasowy, który lepiej odpowiada realistycznym ograniczeniom decyzyjnym. Celem było sprawdzenie, czy agent MCTS + Heavy Playouts osiąga współczynnik zwycięstw powyżej 55% w bezpośrednim starciu z bazowym MCTS.

Limit czasu wynosił 250 milisekund, co pozwoliło utrzymać akceptowalny czas całego eksperymentu przy zachowaniu porównywalnych warunków dla obu agentów. Obaj gracze korzystali z tego samego limitu czasowego. Dodatkowo agent MCTS + Heavy Playouts w fazie symulacji wykorzystywał strategię ϵ -greedy z parametrem $\epsilon = 0.1$, dzięki czemu z prawdopodobieństwem 10% wykonywał losowy legalny ruch, a w pozostałych przypadkach kierował się oceną heurystyczną.

Podobnie jak w przypadku H1, eksperyment przeprowadzono na planszach 6×8 , 7×7 , 8×6 oraz 8×8 . Dla każdego rozmiaru planszy rozegrano 40 partii: 20 z agentem MCTS + Heavy Playouts jako graczem białym oraz 20 z tym samym agentem jako graczem czarnym. Łącznie w celu weryfikacji hipotezy H2 rozegrano 160 partii.

2.2.2 Analiza wyników

Wyniki eksperymentu przedstawiono na Rysunku 10. Są one przeciwne do obserwacji uzyskanych dla hipotezy H1. Wariant wykorzystujący Heavy Playouts zdecydowanie przekroczył próg 55% dla każdego badanego rozmiaru planszy. Najniższą skuteczność osiągnął na planszach 7×7 oraz 8×8 , gdzie współczynnik zwycięstw wyniósł 87.5%. Najlepszy rezultat uzyskano na planszy 6×8 , gdzie agent wygrał 38 z 40 rozgrywek (95.0%). Co istotne, wyraźna przewaga utrzymała się zarówno przy agregacji wyników, jak i przy rozbięciu na kolor kontrolowany przez agenta testowego. Dla każdego rozmiaru planszy agent MCTS + Heavy Playouts osiągał wyniki co najmniej tak dobre po stronie czarnej jak po stronie białej. Największą dysproporcję zaobserwowano dla planszy 8×8 , gdzie współczynnik zwycięstw wyniósł 80% dla białych oraz 95% dla czarnych. Wynik 95% pojawił się również dla czarnych na planszy 8×6 oraz dla obu kolorów na planszy 6×8 .



Rysunek 10: Współczynnik zwycięstw agenta MCTS + Heavy Playouts w starciu z klasycznym MCTS przy limicie 250 ms na ruch.

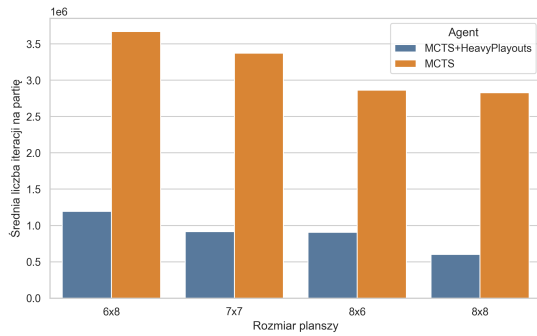
Szczegółowe wyniki liczbowe oraz analizę obciążenia obliczeniowego przedstawiono w Tabeli 9.

Tabela 9: Wyniki eksperymentu H2: MCTS + Heavy Playouts przeciwko klasycznemu MCTS.

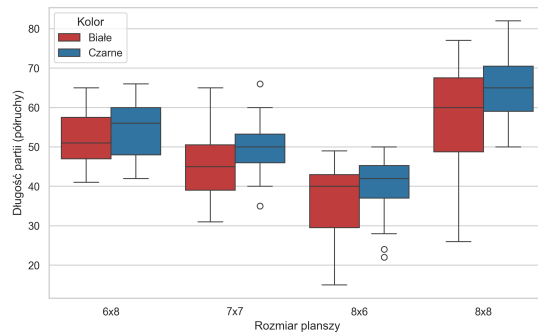
Plansza	Zwycięstwa HP	Skuteczność	Śr. dł. partii	Śr. iteracje (HP / MCTS)
6 × 8	38/40	95.0%	53.4	1.19M / 3.67M
7 × 7	35/40	87.5%	47.8	0.91M / 3.37M
8 × 6	37/40	92.5%	38.1	0.90M / 2.86M
8 × 8	35/40	87.5%	61.0	0.60M / 2.82M
Łącznie	145/160	90.6%	50.1	—

Sumarycznie agent MCTS + Heavy Playouts wygrał 145 ze 160 partii, co daje skuteczność 90.6%. Przedział ufności Wilsona na poziomie 95% wynosił [85.1%, 94.2%]. Jest to przedział estymujący prawdopodobny zakres rzeczywistego współczynnika zwycięstw na podstawie skończonej próby 160 partii. Zastosowanie przedziału Wilsona jest korzystne dla danych procentowych, ponieważ daje stabilniejszą ocenę niepewności niż proste przybliżenie normalne, szczególnie gdy wynik znajduje się blisko 0% lub 100%. W tym przypadku cały przedział leży znacznie powyżej progu 55%, co wzmacnia wniosek o pozytywnej weryfikacji hipotezy.

Kluczową obserwacją w tym eksperymencie jest duża dysproporcja w liczbie iteracji wykonywanych przez oba algorytmy w tym samym oknie czasowym (250 ms). Jak wcześniej wspomniano, wprowadzenie heurystyki do fazy *playout* jest kosztowne obliczeniowo. W efekcie algorytm Heavy Playouts wykonywał średnio od około 3 do 4.7 razy mniej iteracji niż klasyczny wariant MCTS. Dla plansz 7 × 7 oraz 8 × 8 różnica ta była najwyższa (odpowiednio około 3.7 oraz 4.7) i korelowała z nieco niższą skutecznością agenta. Dla plansz 6 × 8 i 8 × 6 relacja wynosiła około 3.1 iteracji klasycznego MCTS na jedną iterację Heavy Playouts.



(a) Średnia liczba iteracji



(b) Rozkład długości partii

Rysunek 11: Dodatkowe charakterystyki eksperymentu H2: koszt obliczeniowy oraz długość partii.

Na wszystkich badanych rozmiarach plansz widoczna jest wyraźna asymetria długości gry w zależności od koloru kontrolowanego przez agenta Heavy Playouts (Rysunek 11b). Partie, w których wariant heurystyczny grał czarnymi, były konsekwentnie dłuższe od partii, w których grał białymi. Jak wspomniano wcześniej, w grze Breakthrough białe posiadają naturalną przewagę pierwszego ruchu. W tym eksperymencie dłuższa rozgrywka po stronie czarnej może jednak wynikać z faktu, że Heavy Playouts skuteczniej opóźniał postęp przeciwnika dzięki trafniejszej ewaluacji defensywnej podczas symulacji. Wydłużało to czas trwania partii, pozwalając agentowi zneutralizować inicjatywę białych i częściej wygrywać w późniejszej fazie gry.

2.2.3 Wnioski końcowe

Hipoteza H2 zostaje przyjęta. Przy limicie 250 ms na ruch agent MCTS + Heavy Playouts osiągnął 90.6% zwycięstw w bezpośrednim starciu z bazowym MCTS, wyraźnie przekraczając wymagany próg 55%. Wyniki eksperymentu wskazują, że w grze Breakthrough podniesienie jakości pojedynczej symulacji poprzez wprowadzenie wiedzy domenowej kompensuje straty wynikające ze zwiększonego czasu obliczeń i mniejszego rozmiaru generowanego drzewa gry. Mimo że agent wykonuje mniej iteracji, informacje płynące z każdej symulacji są bardziej wartościowe. Heurystyka w fazie *playout* lepiej odzwierciedla zachowania istotne strategicznie: bicie pionów przeciwnika, ochronę własnych pionów oraz unikanie niekorzystnych pozycji na krawędziach planszy. Na podstawie tak ukierunkowanych symulacji agent podejmuje decyzje wyraźnie skuteczniejsze niż klasyczny MCTS.

2.3 Hipoteza 3: Przewaga nad ewaluacją statyczną

2.3.1 Metodologia i przebieg eksperymentu

Weryfikacja hipotezy wymagała przeprowadzenia serii rozgrywek turniejowych, w których zautomatyzowani agenci bazujący na algorytmie MCTS zostali skonfrontowani z agentem heurystycznym, wykorzystującym algorytm Minimax ze statyczną funkcją oceny pozycji. Celem eksperymentu było zbadanie, czy probabilistyczne podejście do przeszukiwania przestrzeni stanów pozwala na uzyskanie przewagi nad deterministycznym oponentem o ograniczonym horyzoncie planowania.

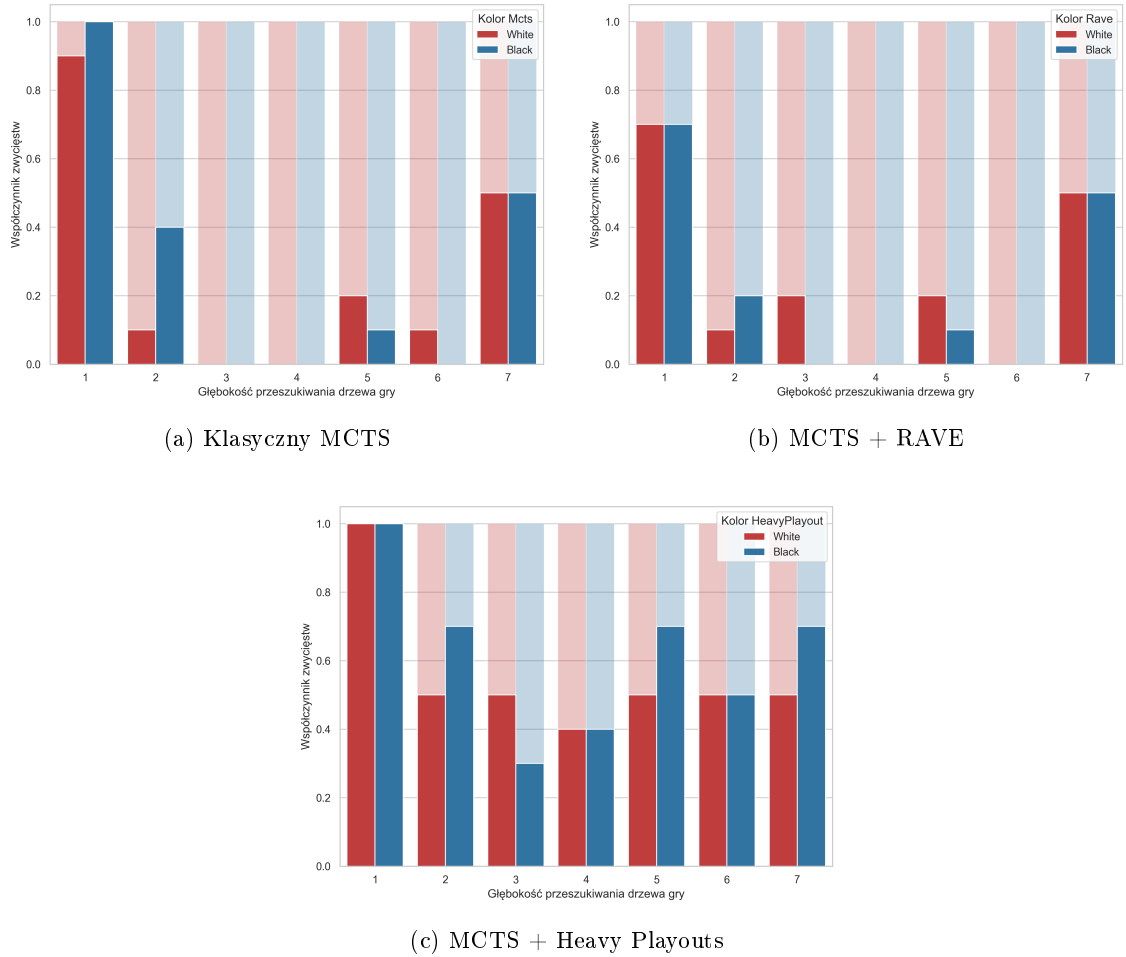
Eksperyment podzielono na trzy główne serie pomiarowe, odpowiadające testowanym wariantom algorytmu sterującego: MCTS, MCTS + RAVE oraz MCTS + Heavy Playouts. W każdej z serii agent testowy (korzystający ze stałego budżetu iteracyjnego 10000) rozgrywał partie przeciwko agentowi Minimax. Siła oponenta była stopniowo zwiększana poprzez inkrementację limitu maksymalnej głębokości przeszukiwania drzewa gry (parametr d) w przedziale od $d = 1$ do $d = 7$. Pozwoliło to na weryfikację zachowania wariantów stochastycznych w starciu z przeciwnikiem o rosnących zdolnościach taktycznych.

Dla każdej testowanej głębokości oponenta rozegrano 20 niezależnych partii. Aby zneutralizować wpływ przewagi pierwszego ruchu, zrezygnowano z przypisywania stałych ról — w 10 partiach agent kontrolował piony białe, a w pozostałych 10 piony czarne. Tym samym każda z trzech serii pomiarowych składała się ze 140 partii, co przełożyło się na 420 rozegranych gier w całym eksperymencie.

Zgodnie z przyjętymi założeniami, główną metryką decydującą o weryfikacji hipotezy H3 był sumaryczny odsetek wygranych partii (wymagany próg powyżej 70%). Dodatkowo, w celu pogłębienia analizy, z zebranych logów wyodrębniono informacje na temat średniej długości trwania rozgrywki (liczby ruchów) oraz czasów namysłu poszczególnych algorytmów.

2.3.2 Analiza wyników

Zestawienie odsetka wygranych partii dla wszystkich trzech wariantów algorytmu MCTS (MCTS, MCTS + RAVE oraz MCTS + Heavy Playouts) w zależności od głębokości przeszukiwania agenta Minimax przedstawiono na Rysunku 12 oraz w Tabeli 10.



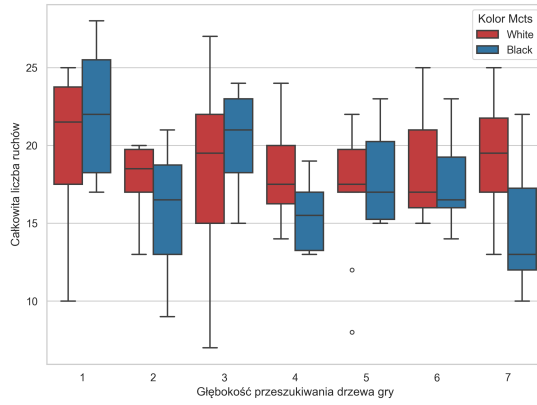
Rysunek 12: Porównanie odsetka wygranych partii w zależności od głębokości przeszukiwania agenta Minimax dla różnych wariantów algorytmu MCTS.

Bezpośrednie porównanie tych wykresów pozwala na sformułowanie kilku obserwacji dotyczących stabilności i siły gry poszczególnych modyfikacji:

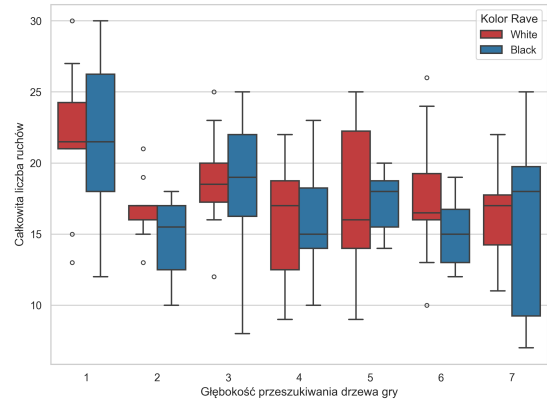
1. Niezależnie od zastosowanego wariantu MCTS, można zaobserwować wyraźny trend spadku skuteczności w początkowej fazie wykresu. Dla najmniejszej głębokości ($d = 1$), wszystkie modyfikacje oparte na symulacjach całkowicie dominują agenta Minimax, osiągając wskaźniki zwycięstw bliskie lub równe 100% ($\geq 70\%$). Jednakże, wraz ze wzrostem limitu głębokości do $d = 3$ i $d = 4$, siła gry Minimaxa znacząco rośnie, co przekłada się na gwałtowny spadek skuteczności agentów probabilistycznych. W tym przedziale klasyczny MCTS oraz MCTS z modyfikacją RAVE przegrywają niemal wszystkie partie, co obnaża podatność losowych oraz pół-losowych symulacji na szerszy horyzont planowania agenta Minimax.
2. Szczególnie interesujące jest zachowanie algorytmów dla dużych głębokości przeszukiwania oponenta ($d \geq 5$). Na wykresach zauważalne jest swoiste odwrócenie trendu — skuteczność wariantów MCTS ponownie zaczyna rosnąć, a dla $d = 7$ agent Minimax zaczyna regularnie przegrywać z ulepszonymi wariantami (Heavy Playouts i RAVE osiągają lepsze wyniki w tym rejonie w porównaniu do klasycznego MCTS, choć i on odnotowuje wzrost do poziomu około 50%). Przy bardzo dużych głębokościach (np. $d = 7$), agent Minimax staje się ofiarą własnego determinizmu — „ślepo” ufa zaprogramowanym wagom heurystycznym. Warianty MCTS, dzięki pełnym symulacjom do końca partii, potrafią elastyczniej identyfikować rzeczywiste szanse na wygraną.
3. Spośród testowanych modyfikacji, wariant MCTS + Heavy Playouts (Rysunek 12c) wykazał się największą odpornością na siłę taktyczną Minimaxa w środkowym przedziale głębokości ($d = 3, d = 4$).
4. RAVE (Rysunek 12b) zachowywało się w sposób zbliżony do klasycznego MCTS.

Tabela 10: Odsetek wygranych partii przez różne warianty algorytmu MCTS w starciu z algorytmami Minimax o zmiennej głębokości przeszukiwania (d).

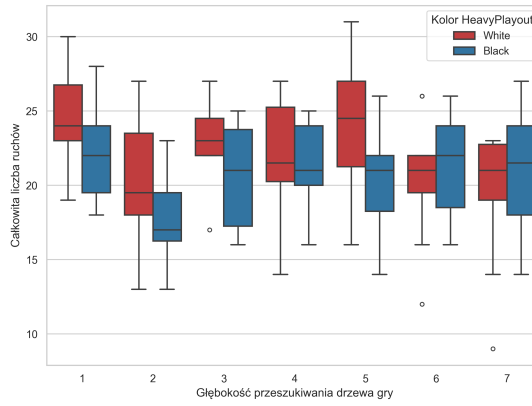
Głębokość	MCTS	MCTS + RAVE	MCTS + Heavy Playouts
1	95.0%	70.0%	100.0%
2	25.0%	15.0%	60.0%
3	0.0%	10.0%	40.0%
4	0.0%	0.0%	40.0%
5	15.0%	15.0%	60.0%
6	5.0%	0.0%	50.0%
7	50.0%	50.0%	60.0%



(a) Klasyczny MCTS



(b) MCTS + RAVE



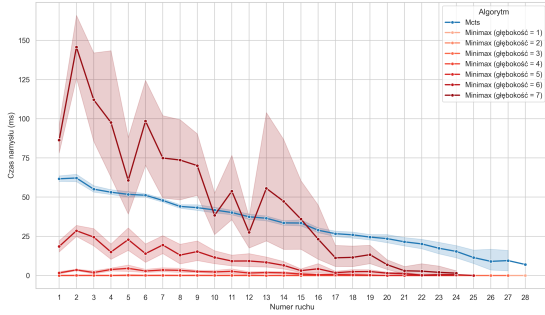
(c) MCTS + Heavy Playouts

Rysunek 13: Zestawienie dystrybucji całkowitej liczby ruchów rozegranych w partiach przeciwko agentowi Minimax dla różnych wariantów algorytmu MCTS.

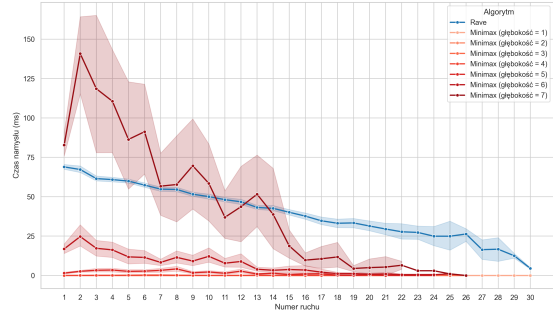
Kolejnym elementem poddanym analizie była średnia długość rozegranych partii, wyrażona jako całkowita liczba ruchów wykonana przez obu graczy przed osiągnięciem stanu końcowego (Rysunek 13, Tabela 11). Wykresy uwzględniają odrębne „pudełka” dla sytuacji, w której agent MCTS steruje bierkami białymi oraz czarnymi.

Tabela 11: Średnia długość partii (wyrażona jako całkowita liczba ruchów obu graczy) wraz z odchyleniem standardowym, w starciach wariantów MCTS z algorytmem Minimax.

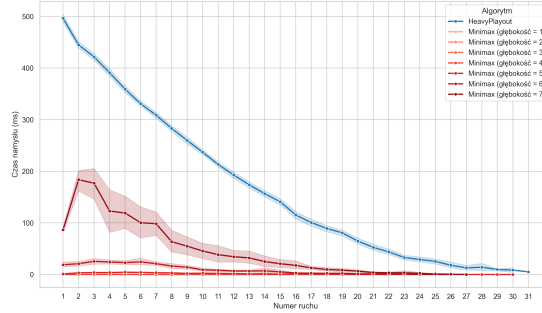
Głębokość	MCTS	MCTS + RAVE	MCTS + Heavy Playouts
1	21.1 ± 4.3	21.5 ± 5.5	23.4 ± 3.5
2	17.0 ± 3.2	15.8 ± 2.6	19.1 ± 3.9
3	19.6 ± 4.7	18.6 ± 4.5	21.8 ± 3.4
4	16.9 ± 2.9	16.1 ± 4.1	21.8 ± 3.3
5	17.4 ± 3.5	17.4 ± 3.9	22.1 ± 4.2
6	17.9 ± 3.1	16.3 ± 3.9	21.0 ± 3.8
7	17.1 ± 4.7	15.8 ± 4.9	20.2 ± 4.4



(a) Klasyczny MCTS



(b) MCTS + RAVE



(c) MCTS + Heavy Playouts

Rysunek 14: Średni czas namysłu (w milisekundach) dla różnych wariantów algorytmu MCTS oraz ich przeciwników (agenta Minimax o rosnącej głębokości przeszukiwania).

Ostatnim, w ramach tego eksperymentu, badanym elementem była analiza średniego czasu namysłu wymaganego do wygenerowania pojedynczego ruchu. Profile wydajnościowe, zestawione na układzie wykresów (Rysunek 14), obrazują różnice w naturze obu podejść do przeszukiwania przestrzeni stanów.

Przewidywalną obserwacją, jest wykładniczy wzrost czasu namysłu algorytmu Minimax wraz ze wzrostem limitu głębokości przeszukiwania (d). Dla płytkich wartości ($d \leq 4$), decyzje podejmowane są niemal natychmiastowo, jednakże przekroczenie progu $d = 5$ powoduje drastyczny wzrost wielkości drzewa gry (wynikające z wysokiego współczynnika rozgałęzienia w gry). Dla maksymalnej testowanej głębokości ($d = 7$), pojedynczy ruch oponenta statycznego zajmuje znacząco więcej czasu niż iteracyjne rozwijanie drzewa przez warianty MCTS oraz MCTS + RAVE, co pokazuje ograniczenia skalowalności naiwnego (bądź wyposażonego jedynie w cięcia alfa-beta) przeglądu zupełnego.

Ciekawszym zjawiskiem jest zachowanie czasu namysłu dla algorytmów opartych na MCTS. Mimo iż operowały one na stałym, odgórnie narzuconym limicie całkowitej liczby iteracji w każdym ruchu (niezależnie od przeciwnika), czas potrzebny na realizację tego budżetu skraca się (w przybliżeniu liniowo) wraz z rozwojem gry. Losowe lub pół-losowe rozgrywki algorytmów MCTS trwają coraz krócej ponieważ pozostało niewiele ruchów do zakończenia rozgrywki.

Analizując warianty MCTS między sobą, można zauważyć, że zastosowanie heurystyki RAVE nie wprowadziło znaczącego narzutu obliczeniowego, natomiast wielokrotne wywoływanie funkcji oceny pozycji w wariantcie Heavy Playouts, dosyć znacząco wydłuża czas namysłu algorytmu, którego średni czas namysłu do 11 ruchu przekracza 200 ms (przy stałym limicie 10000 iteracji).

2.3.3 Wnioski końcowe

Analiza zebranych danych pozwala stwierdzić, że dla najbardziej podstawowego wariantu płytkiego przeszukiwania, czyli dla głębokości $d = 1$, wszystkie testowane modyfikacje algorytmu stochastycznego z sukcesem osiągnęły lub przekroczyły wymagany próg skuteczności. Klasyczny wariant MCTS uzyskał wynik 95.0%, MCTS z heurystyką RAVE osiągnął barierę 70.0%, natomiast wariant kierowany (Heavy Playouts) zdominował przeciwnika całkowicie, wygrywając 100.0% rozegranych partii.

Hipoteza H3 zostaje przyjęta. Należy jednak zaznaczyć, że założony próg 70% wygranych jest spełniony wyłącznie dla głębokości równej 1. Zwiększenie horyzontu planowania agenta Minimax zaledwie o jeden lub dwa półruchy ($d \geq 2$) powodowało gwałtowne załamanie skuteczności metod opartych o algorytm MCTS.

2.4 Hipoteza 4: Wpływ rozmiaru planszy na asymetrię szans

2.4.1 Metodologia i przebieg eksperymentu

Weryfikacja czwartej hipotezy badawczej (H4) koncentrowała się na ocenie wpływu geometrii planszy na asymetrię szans w grze. Ze względu na specyfikę mechaniki Breakthrough, w której celem jest dotarcie do linii końcowej przeciwnika, gracz wykonujący pierwszy ruch (białe) dysponuje naturalną przewagą tempa. Hipoteza zakładała, że wraz ze wzrostem wymiarów planszy przewaga koloru białego ulegnie marginalizacji na rzecz strategii. Zgodnie z hipotezą, zmodyfikowane algorytmy MCTS będą w stanie odnosić relatywnie częstsze zwycięstwa z pozycji czarnych na większych planszach.

Rozgrywki przeprowadzono dla asymetrycznych wymiarów planszy. Szerokość ustalono jako stałą wartość $W = 8$, natomiast wysokość (H) stanowiła zmienną niezależną przyjmującą wartości z rosnącego zbioru: $H \in \{5, 6, 7, 8, 9, 10, 11\}$.

Eksperyment podzielono na trzy symetryczne serie pomiarowe, w których po obu stronach planszy stawali identyczni agenci:

1. **Seria 1:** Klasyczny algorytm MCTS (UCT) grający przeciwko sobie.
2. **Seria 2:** Wariant MCTS + RAVE.
3. **Seria 3:** Wariant MCTS + Heavy Playouts.

Aby zapewnić agentom budżet adekwatny do rozmiaru gry, maksymalna liczba iteracji zmieniała się liniowo w zależności od wysokości planszy zgodnie ze wzorem:

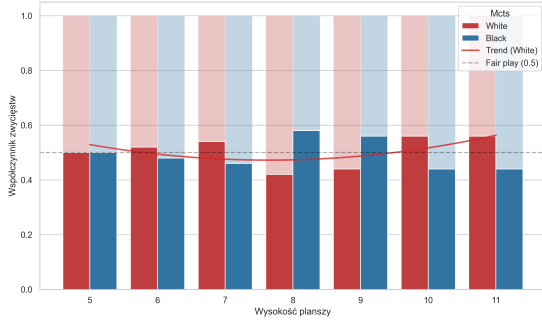
$$I = 2000 \cdot H$$

Tym samym dla planszy 8×5 limit wynosił 10000 iteracji, a dla 8×11 już 22000.

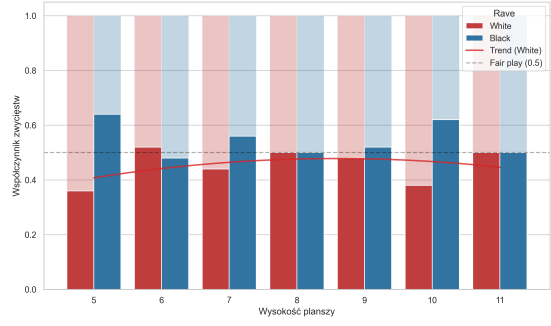
Dla każdego z testowanych wymiarów planszy i dla każdej z trzech serii rozegrano dokładnie 50 partii turniejowych. Ponieważ w każdej serii agenci reprezentowali identyczną siłę gry, zrezygnowano z rotacji kolorów na rzecz jednorodnego monitorowania współczynnika zwycięstw z perspektywy białego gracza.

2.4.2 Analiza wyników

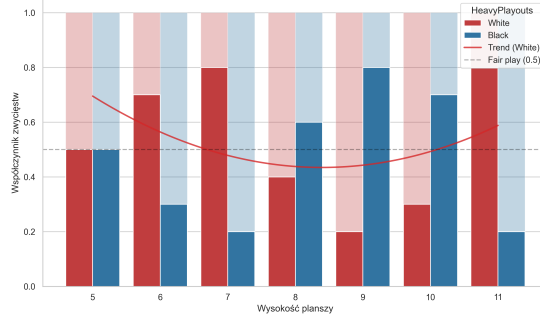
Zgromadzone dane pozwoliły na prześledzenie zachowania wskaźnika zwycięstw dla gracza rozpoczynającego (Białe) w zależności od wysokości planszy H . Zestawienie wyników dla wszystkich trzech symetrycznych par algorytmów (Klasyczny MCTS, MCTS + RAVE, MCTS + Heavy Playouts) przedstawiono w formie wykresów na Rysunku 15. Szczegółowe dane liczbowe zostały zgromadzone w Tabeli 12.



(a) Klasyczny MCTS



(b) MCTS + RAVE



(c) MCTS + Heavy Playouts

Rysunek 15: Odsetek zwycięstw gracza rozpoczynającego (Białe) w starciach symetrycznych w zależności od wysokości planszy (H) przy stałej szerokości $W = 8$.

Wstępna analiza wyników wskazuje na dużą zmienność (wariancję) wskaźnika wygranych w poszczególnych wariantach na szerokim przedziale wysokości planszy. W celu ułatwienia interpretacji tych odchyleń, na wykresach naniesiono linię trendu obrazującą ogólną tendencję zmian skuteczności (modelowaną przez wielomian stopnia 2), a także poziomą linię wyznaczającą próg 50% (tzw. linię *fair play*), która reprezentuje stan idealnej równowagi szans między stroną rozpoczynającą a broniącą.

Analiza zgromadzonych danych pozwala na pozytywne zweryfikowanie postawionych warunków. Rozpatrując kluczowy przedział wysokości od $H = 7$ do $H = 9$, zaobserwowano spadek odsetka wygranych Białego. Dla klasycznego MCTS skuteczność Białych spadła z poziomu 54.0% ($H = 7$) do 42.0% ($H = 8$) oraz 44.0% ($H = 9$), co oznacza utratę inicjatywy startowej i przejście inicjatywy (choć z niewielką przewagą) na stronę Czarnych.

Wprowadzenie wiedzy domenowej poprzez modyfikację Heavy Playouts ujawniło wyraźny trend malejący: od dominacji Białych wynoszącej 80.0% na planszy 8×7 , poprzez spadek do 40.0% na planszy 8×8 , aż do zaledwie 20.0% na planszy 8×9 .

Należy zaznaczyć, że trend ten nie zachowuje idealnej liniowości w nieskończoność (co pokazują anomalie dla dużej planszy $H = 11$), a wariant RAVE charakteryzował się większą zachowawczością wyników wokół linii remisu (50%). Nie zmienia to jednak faktu, że w zakładanym w kryteriach przedziale badawczym (odpowiednik plansz średnich i dużych) zjawisko marginalizacji przewagi Białych jest zauważalne.

Tabela 12: Zestawienie skuteczności i średniej długości partii dla wariantów MCTS w zależności od wysokości planszy.

Wariant	Wysokość planszy	Współczynnik zwycięstw (Białe/Czarne)	Średnia długość partii
Klasyczny MCTS	5	50/50%	15.8 ± 3.5
	6	52/48%	20.4 ± 4.1
	7	54/46%	25.4 ± 6.2
	8	42/58%	29.1 ± 7.7
	9	44/56%	31.3 ± 7.2
	10	56/44%	39.2 ± 8.4
	11	56/44%	38.5 ± 10.7
MCTS + Heavy Playouts	5	50/50%	16.3 ± 1.7
	6	70/30%	19.8 ± 4.7
	7	80/20%	27.5 ± 2.6
	8	40/60%	34.9 ± 5.1
	9	20/80%	40.7 ± 4.0
	10	30/70%	52.7 ± 6.4
	11	80/20%	56.5 ± 3.8
MCTS + RAVE	5	36/64%	15.2 ± 3.2
	6	52/48%	18.4 ± 4.1
	7	44/56%	19.7 ± 4.3
	8	50/50%	23.0 ± 5.2
	9	48/52%	25.6 ± 5.9
	10	38/62%	28.6 ± 7.0
	11	50/50%	30.9 ± 8.5

2.4.3 Wnioski końcowe

Hipoteza 4 zostaje przyjęta. Rozmiar planszy w grze Breakthrough w istotne znaczenie na balans rozgrywki, a na rozszerzonych planszach przemyślana strategia (reprezentowana przez ukierunkowane symulacje) pozwala na statystycznie istotne przejście kontroli nad partią przez gracza broniącego (Czarne).

3 Testy z udziałem ludzi

3.1 Metodologia i przebieg badania

Ostatnim etapem ewaluacji była seria rozgrywek z udziałem ludzi. Celem tego eksperymentu była analiza tego, jak agent radzi sobie z przeciwnikiem, który nie gra według deterministycznego skryptu, może świadomie wykorzystywać słabości zauważone w trakcie rozgrywki oraz adaptować się do stylu gry przeciwnika.

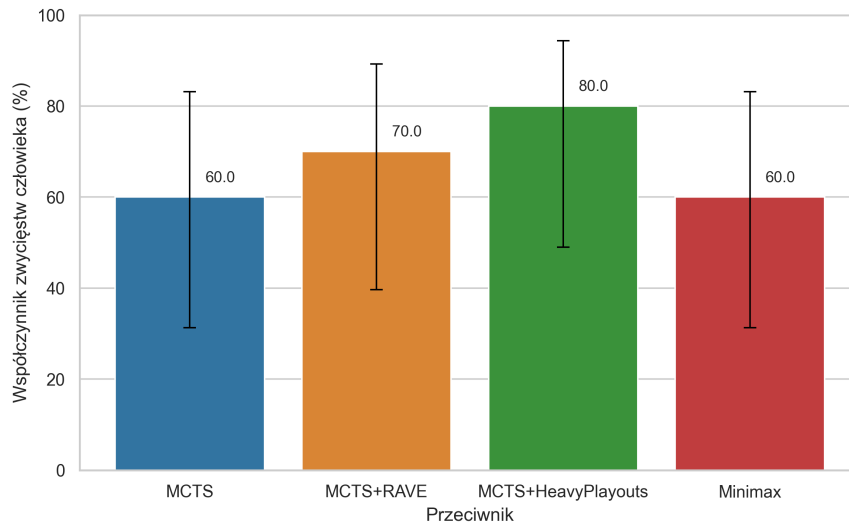
W badaniu wzięło udział pięciu testerów, którzy nie byli twórcami agentów. Każdy z nich rozegrał osiem partii: po jednej przeciwko każdemu z agentów na planszy 7×7 oraz po jednej na planszy 8×8 . Łącznie zebrano więc 40 próbek. Algorytmy były następujące:

1. klasyczny MCTS,
2. MCTS z heurystyką RAVE ze stałą $k = 100$,
3. MCTS z Heavy Playouts,
4. Minimax o głębokości przeszukiwania $d = 8$.

W rozgrywkach na planszy 7×7 człowiek grał kolorem białym, a na planszy 8×8 kolorem czarnym. Kolejność konfiguracji była zapisana osobno dla każdego uczestnika w plikach `pXX_order.csv`. Na końcu każdej rozgrywki uczestnicy badania otrzymywali dwa pytania: o poziom trudności oraz o naturalność strategii przeciwnika i oceniali je w skali od 1 do 5.

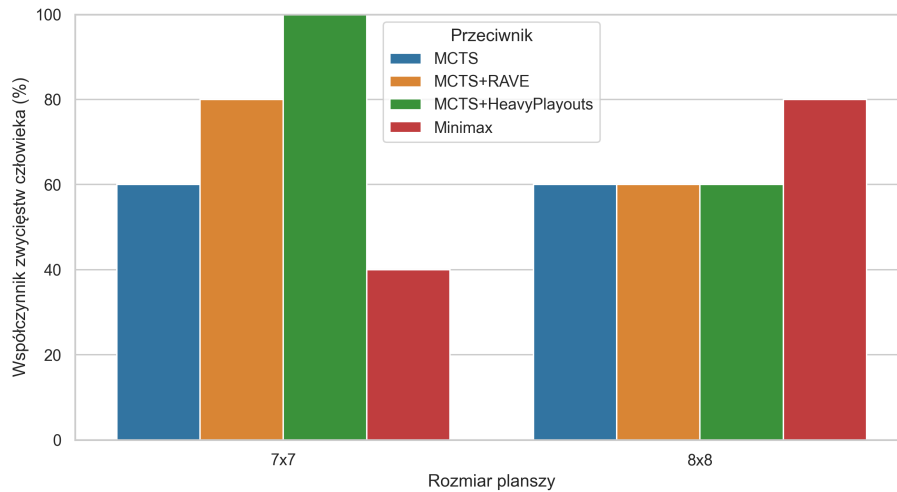
3.2 Analiza wyników

Zestawienie wyników ilościowych przedstawiono na Rysunkach 16–20. W całym badaniu gracze ludzcy wygrali 27 z 40 rozegranych partii, co odpowiada skuteczności 67.5%.



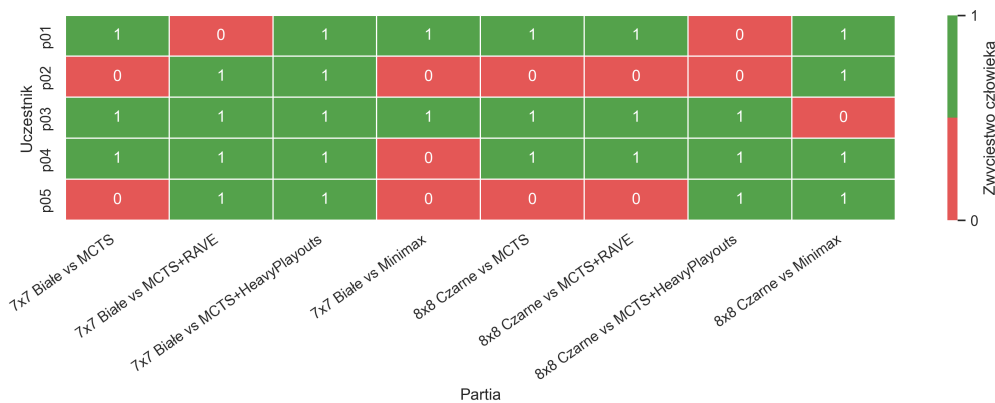
Rysunek 16: Współczynnik zwycięstw ludzi w zależności od wariantu przeciwnika. Słupki błędów oznaczają przedziały ufności Wilsona.

Najwięcej problemów ludziom sprawiały klasyczny MCTS oraz Minimax, przeciwko którym gracze uzyskali po 60.0% zwycięstw. Najłatwiejszym wariantem w bezpośredniej rywalizacji z ludźmi okazał się MCTS + Heavy Playouts. Testerzy wygrali z nim aż 80.0% partii.



Rysunek 17: Współczynnik zwycięstw ludzi w podziale na rozmiar planszy oraz wariant przeciwnika.

Podział względem rozmiaru planszy wskazuje, że ludzie wygrali 14 z 20 partii na planszy 7×7 oraz 13 z 20 partii na planszy 8×8 . Oznacza to, że zmiana strony i rozmiaru planszy nie wpłynęła znacząco na sam wynik rozgrywki. Różnice były natomiast widoczne dla konkretnych agentów. Na planszy 7×7 Minimax wygrał z ludźmi 3 z 5 partii, natomiast na planszy 8×8 ludzie pokonali go w 4 z 5 partii. Może to sugerować, że na małej planszy taktyczny charakter Minimaxa był trudniejszy do zatrzymania: agent znajdował ścieżki prowadzące piony do celu, które umykały badanym. Na większej planszy gracze mieli więcej przestrzeni do formowania struktur oraz lepszego zarządzania pionami.



Rysunek 18: Macierz wyników poszczególnych uczestników. Wartość 1 oznacza zwycięstwo człowieka, natomiast 0 zwycięstwo agenta.

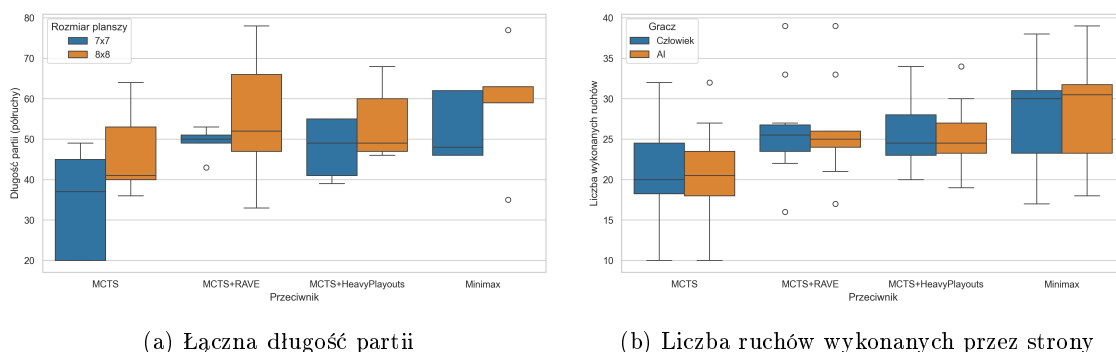
Macierz wyników pokazuje dużą zmienność między uczestnikami. Mimo że każdy uczestnik został przeszkolony przed pierwszą rozgrywką, widoczna jest wyraźna dysproporcja pomiędzy badanymi. Jeden z testerów wygrał 7 z 8 partii, podczas gdy inny wygrał jedynie 3 z 8 partii. Gracze, którzy szybko zauważali podatność agentów MCTS na przepuszczanie wysuniętych pionów, byli w stanie regularnie wykorzystywać tę słabość. Z kolei gracze mniej konsekwentnie broniący wszystkich kolumn częściej dopuszczali do sytuacji, w której agent sam wymijał ich strukturę pionów.

Tabela 13: Zestawienie wyników ludzi przeciwko poszczególnym wariantom AI.

Przeciwnik	Wygrane ludzi	Wygrane AI	Skuteczność ludzi	Śr. długość partii
MCTS	6/10	4/10	60.0%	40.5
MCTS + RAVE	7/10	3/10	70.0%	52.2
MCTS + Heavy Playouts	8/10	2/10	80.0%	50.9
Minimax	6/10	4/10	60.0%	56.1

3.3 Długość partii i zachowanie materiału

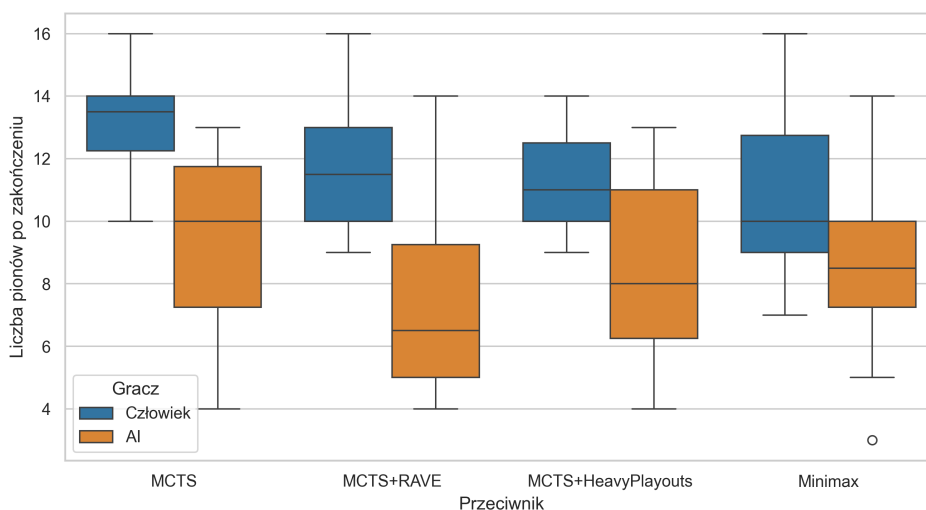
Oprócz samego wyniku przeanalizowano również liczbę wykonanych ruchów oraz liczbę pionów pozostających na planszy po zakończeniu partii. Metryki te pozwalają wnioskować, jak szybko przełamywana była linia przeciwnika.



Rysunek 19: Porównanie długości partii oraz liczby ruchów wykonanych przez człowieka i agenta.

Najkrótsze partie występowały przeciwko klasycznemu MCTS, gdzie średnia długość wyniosła 40.5 póhruchu. Partie przeciwko Minimaxowi były najdłuższe i trwały średnio 56.1 póhruchu. Wynik ten jest zgodny z obserwacjami jakościowymi: Minimax często grał bardziej zwartą strukturą, przesuwając większą część linii pionów do przodu i rzadziej natychmiastowo odsłaniał pojedynczą kolumnę. W efekcie partia wymagała od człowieka dłuższego szukania przełamania lub obrony. Klasyczny wariant MCTS często bez widocznego uzasadnienia oddawał swoje pionki w środkowych kolumnach. Następnie próbował zapełnić powstałą lukę pionami z bocznych stref planszy, co odsłaniało skrzydła i pozwalało badanym kończyć partie właśnie tymi kanałami.

W przypadku wariantów MCTS liczba ruchów człowieka i agenta była bardzo zbliżona, co wynika z naprzemiennego charakteru gry. Różnice można jednak zauważyć na Rysunku 20, który przedstawia liczbę pionów pozostających na planszy po zakończeniu partii.

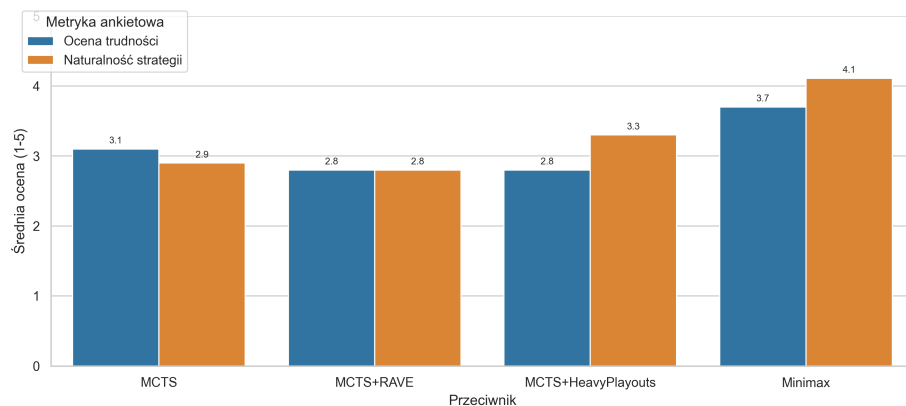


Rysunek 20: Liczba pionów człowieka i agenta pozostających na planszy po zakończeniu partii.

We wszystkich wariantach końcowa przewaga materialna była po stronie ludzi. Średnio po partiach przeciwko MCTS człowiek zachowywał 13.2 pionu, a agent 9.3 pionu. Dla MCTS + RAVE wartości te wynosiły odpowiednio 11.9 oraz 7.4 pionu, dla MCTS + Heavy Playouts 11.3 oraz 8.2 pionu, natomiast dla Minimaxa 10.8 oraz 8.4 pionu. Przyczyną takiej dysproporcji może być to, że w pozycjach przegranych agenci poświęcali pionu, starając się opóźnić porażkę. Badani, mimo że często mogli już bezpośrednio wygrać partię, zbijali podstawiane pionu, budując dodatkową przewagę materialną. Dysproporcja ta sugeruje również, że zwycięstwa ludzi nie były wyłącznie efektem pojedynczego szybkiego przebicia lub sprytnego ruchu, lecz często wiązały się ze stopniowym budowaniem przewagi strukturalnej i materialnej.

3.4 Ocena ankietowa i obserwacje jakościowe

Uczestnicy oceniali trudność oraz naturalność ruchów przeciwników w skali od 1 do 5. Zestawienie średnich ocen przedstawiono na Rysunku 21. Najwyżej ocenionym wariantem w obu kategoriach był Minimax. Średnia ocena trudności wyniosła dla niego 3.7, a naturalności strategii 4.1. Klasyczny MCTS uzyskał średnią trudność 3.1 i naturalność 2.9, MCTS + RAVE odpowiednio 2.8 i 2.8, natomiast MCTS + Heavy Playouts 2.8 i 3.3.



Rysunek 21: Średnie oceny ankietowe przeciwników AI w skali od 1 do 5.

Oceny ankietowe są spójne z feedbackiem testerów oraz z obserwacjami prowadzonymi podczas rozgrywek. Minimax był odbierany jako przeciwnik najbardziej agresywny i najbardziej planowy. Testerzy zwracali uwagę, że często przesunął całą linię pionów w sposób symetryczny, przez co wymuszał analizę kilku możliwych przełamień jednocześnie. Na planszy 8×8 przeciętny badany miał trudność z konsekwentnym sprawdzaniem wszystkich ruchów do przodu, podczas gdy Minimax systematycznie przeszukiwał takie możliwości.

Warianty MCTS były oceniane bardziej ambiwalentnie. Testerzy zauważali, że potrafiły dobrze bronić zastawione pułapki oraz reagować na przenoszenie pionów na jedną stronę planszy. Wielokrotnie udawało im się ominąć strukturę obronną zbudowaną przez badanego. Często pojawiały się też podobne schematy gry, na przykład ustawianie dwóch pionów jeden za drugim w jednej kolumnie. Dzięki takiemu ustawieniu bicie lub łatwe ominięcie struktury staje się trudniejsze. Z drugiej strony klasyczny MCTS oraz MCTS + RAVE miały tendencję do zbyt odważnego wypychania najbardziej wysuniętego pionu, co w praktyce prowadziło do oddawania materiału. Według autorów prawdopodobną przyczyną jest losowy charakter fazy *playout*: w symulacjach takie zagranie nie zawsze kończy się natychmiastową karą, ponieważ zabicie pionu jest tylko jedną z wielu losowych możliwości, a czasem wysunięty pion może przypadkowo doprowadzić do zwycięstwa, zaburzając statystyki drzewa.

Ostatnią obserwacją zgłoszoną w ankietach było to, że gdy agent znajdował się w sytuacji trudnej do obrony, często zaczynał wykonywać ruchy poświęcające materiał w celu opóźnienia porażki. Z perspektywy człowieka wyglądało to czasem jak desperackie oddawanie pionów, ale jest to naturalna konsekwencja funkcji celu agenta. Wpływ tego zachowania na liczbę pionów pozostałych na planszy po zakończeniu rozgrywki został opisany wcześniej.

3.5 Wnioski końcowe

Testy z udziałem ludzi pokazały, że skuteczność agentów w rozgrywkach automatycznych nie przekłada się bezpośrednio na odbiór ich siły przez badanych. Pomimo tego, że Minimax osiągał mia-

dząco najlepsze wyniki w poprzednich eksperymentach, a gracze z najlepszym współczynnikiem zwycięstw przegrywali pojedyncze partie tylko z nim, całościowo nie osiągnął on ewidentnie najlepszego wyniku procentowego przeciwko ludziom. Wynika to z jego deterministycznego, taktycznego charakteru gry: nawet jeśli popełniał błędy wynikające z ograniczonej funkcji heurystycznej, jego ruchy częściej sprawiały wrażenie elementów spójnego planu. Był to jednak wariant, który nie wybacał błędów i jeśli znajdował sekwencję ruchów prowadzącą do zwycięstwa, konsekwentnie ją realizował.

Warianty MCTS, zwłaszcza bez Heavy Playouts, częściej ujawniały zachowania trudne do uznania za ludzkie: nadmierne wysuwanie pojedynczych pionów, lokalne oddawanie materiału oraz brak wyraźnej symetrii w strukturze. Strategie te nie były jednak całkowicie losowe, co potwierdzają spostrzeżenia badanych. Badanie z ludźmi wskazuje, że stabilność gry, reakcja na błędy przeciwnika oraz subiektywne wrażenie planowości nie muszą bezpośrednio pokrywać się z końcowym wynikiem partii. Ostatecznie w grze liczy się zwycięstwo, ale przy ocenie agenta warto pamiętać również o tym, jak jego styl jest odbierany przez człowieka. Nie można też zapominać, że próba była bardzo mała, a różnice w odsetkach zwycięstw między wariantami są ograniczone.

4 Podsumowanie

Celem niniejszego projektu była implementacja, optymalizacja oraz ewaluacja algorytmu Monte-Carlo Tree Search (MCTS) wraz z jego modyfikacjami w środowisku deterministycznej gry planszowej Breakthrough. W tym celu stworzono w pełni funkcjonalny system w języku Rust, obejmujący silnik obliczeniowy do porównywania różnych wariantów algorytmu w zautomatyzowanych turniejach self-play oraz graficzny interfejs użytkownika przeznaczony do testów z udziałem ludzi.

Przeprowadzone eksperymenty pozwoliły zweryfikować cztery wcześniej postawione hipotezy oraz sformułować wnioski dotyczące przeszukiwania przestrzeni stanów w grze Breakthrough. Weryfikacja hipotezy H1 pokazała, że samo zastosowanie heurystyki RAVE (AMAF) nie przyniosło oczekiwanej poprawy jakości gry. Ze względu na taktyczny charakter Breakthrough kolejność ruchów oraz tempo dojścia do linii końcowej mają duże znaczenie, dlatego kluczowe założenie RAVE o względnej niezależności wartości ruchu od momentu jego wykonania okazało się w tym przypadku zbyt silne.

Odmienne rezultaty przyniosła weryfikacja hipotezy H2. Wprowadzenie wiedzy domenowej do fazy symulacji wyraźnie podniosło skuteczność agenta w starciu z klasycznym wariantem. Poprawa ta wystąpiła mimo istotnego narzutu obliczeniowego powodowanego przez Heavy Playouts. Wskazuje to, że podniesienie jakości pojedynczej symulacji może kompensować straty wynikające ze zwiększonego czasu obliczeń i mniejszego rozmiaru generowanego drzewa gry.

Starcia wariantów probabilistycznych z deterministycznym algorytmem Minimax w ramach H3 ukazały słabość metod statystycznych przy średnich horyzontach planowania oponenta. Jednocześnie ponownie widoczna była przewaga MCTS + Heavy Playouts nad klasycznym wariantem oraz RAVE, ponieważ był to jedyny wariant MCTS, który nie odstawał wyraźnie od Minimaxa. Badania nad geometrią planszy w ramach H4 pokazały natomiast, że wydłużenie dystansu między graczami skutecznie ogranicza asymetrię szans wynikającą z przewagi pierwszego ruchu białych.

Istotnych obserwacji dostarczyły również testy z udziałem ludzkich graczy. Pokazały one różnicę między skutecznością agenta w starciach z innymi wariantami AI a jego skutecznością w grze przeciwko człowiekowi. Paradoksalnie, agent MCTS + Heavy Playouts, który był zdecydowanym zwycięzcą turnieju różnych wariantów MCTS, okazał się najłatwiejszy do pokonania przez ludzi. Najtrudniejszymi przeciwnikami były natomiast Minimax, tworzący spójne struktury defensywne i logiczne sekwencje bić, oraz klasyczny MCTS. Może to sugerować, że heurystyki optymalizowane pod kątem ulepszania symulacji generują luki taktyczne, które ludzka intuicja potrafi stosunkowo szybko zidentyfikować i wykorzystać.

Podsumowując, zastosowanie metod opartych na Monte-Carlo Tree Search w grze Breakthrough stanowi skuteczne rozwiązanie, choć jego efektywność silnie zależy od sposobu wykorzystania wiedzy domenowej. Kolejnym kierunkiem badań mogłoby być łączenie kilku modyfikacji algorytmu lub jego poszczególnych faz, w szczególności połączenie Heavy Playouts z RAVE oraz wykorzystanie zrównoleglenia obliczeń. Interesującym obszarem dalszych analiz byłaby także modyfikacja funkcji heurystycznej tak, aby lepiej odzwierciedlała taktyki wykorzystywane przez silnych graczy. Najlepsze wyniki w eksperymentach automatycznych zapewniał wariant Minimax, jednak stworzenie agenta, który nie tylko optymalizuje współczynnik zwycięstw, lecz także prezentuje styl gry wymagający od człowieka naturalnej i elastycznej odpowiedzi, pozostaje otwartym wyzwaniem.

Literatura

- [1] *The Rust Programming Language – Installation*, <https://doc.rust-lang.org/book/ch01-01-installation.html#installation> (dostęp 28.01.2026r.).
- [2] *What is rustc?*, <https://doc.rust-lang.org/stable/rustc/index.html#what-is-rustc> (dostęp 28.01.2026r.).
- [3] *The Cargo Book – Installation*, <https://cargo-book.rust.net/en-us/getting-started/installation.html> (dostęp 28.01.2026r.).